Module 3: Data Link Layer and Wireless Networks

Module 3: Data Link Layer and Wireless Networks

Data-Link Layer

- Data Link Control (DLC)
- Multiple Access Protocols (MAC)
- Link-Layer Addressing
- Ethernet Protocol
- Connecting Devices

Wireless Networks

- Wireless LANs
- Mobile IP

Hands-On

- Datalink Provider Interface
- SOCK_PACKET
- PF_PACKET

Overview

- The TCP/IP suite defines protocols mainly for the Application, Transport, and Network layers.
- The Data-Link and Physical layers are implemented by the underlying networks.
- These networks provide communication services to the upper layers.
- Both wired and wireless technologies operate at these lower layers.
- This module focuses on Data-Link Layer concepts and Wired Networks.

Linear Block Codes

- Most practical block codes belong to a special category called **Linear Block Codes**.
- Linear block codes are widely used because:
  - Easy to analyze mathematically
  - Easy to implement in hardware and software
  - Efficient for error detection and correction
- Nonlinear block codes are rarely used because their structure is more complex.

Informal Definition

A block code is linear if the XOR (modulo-2 addition) of any two valid codewords produces another valid codeword.

Linear Block Codes and XOR Operation

Modulo-2 Addition (XOR)

$$0 \oplus 0 = 0$$

$$0 \oplus 1 = 1$$

$$1 \oplus 0 = 1$$

$$1 \oplus 1 = 0$$

Linearity Property

If C_1 and C_2 are valid codewords,

$$C_1 \oplus C_2$$

must also be a valid codeword.

Example 5.5: Linear Block Code

Consider the code from Table 5.1:

Dataword	**Codeword**
00	000
01	011
10	101
11	110

This code is a **Linear Block Code** because XORing any two valid codewords produces another valid codeword.

Example 5.5: Verification Using XOR

$$\begin{array}{r} 011 \\ \oplus\ 101 \\ \hline 110 \end{array}$$

- 011 and 101 are valid codewords.
- XOR result = 110.
- 110 is also a valid codeword.

Linearity Verified

Valid Codeword $\oplus$ Valid Codeword = Valid Codeword

Why Linear Block Codes are Important

- Simplifies encoding and decoding.
- Enables efficient error detection and correction.
- Forms the basis for many practical codes:
  - Parity Codes
  - Hamming Codes
  - CRC Codes
- Widely used in modern communication systems and computer networks.

Key Property

For a Linear Block Code:

$$\text{Valid Codeword} \oplus \text{Valid Codeword} = \text{Valid Codeword}$$

Minimum Distance for Linear Block Codes

- For a linear block code, finding the minimum Hamming distance is very simple.
- The minimum Hamming distance is equal to the smallest number of 1s present in any nonzero valid codeword.

Rule

For a linear block code,

$$d_{\min} = \text{Minimum weight of all nonzero codewords}$$

where **weight** = number of 1s in a codeword.

Example 5.6

Consider the valid codewords from Table 5.1:

Codewords
000
011
101
110

Ignore the all-zero codeword and count the number of 1s:

$$011 \rightarrow 2$$

$$101 \rightarrow 2$$

$$110 \rightarrow 2$$

Example 5.6: Finding $d_{\min}$

Weights of nonzero codewords:

$$w(011) = 2$$

$$w(101) = 2$$

$$w(110) = 2$$

Minimum weight:

$$\min(2, 2, 2) = 2$$

Therefore,

$$\boxed{d_{\min} = 2}$$

Conclusion

The minimum Hamming distance of this linear block code is 2.

Parity-Check Code

- Parity-check code is the simplest and most widely used error-detecting code.
- It is a **linear block code**.
- For a k-bit dataword, one extra bit called the **parity bit** is added.
- Thus, codeword length becomes:

$$n = k + 1$$

- The parity bit is chosen so that the total number of 1s in the codeword is **even**.

Key Property

Minimum Hamming Distance:

$$d_{\min} = 2$$

Hence, parity-check code can detect all single-bit errors.

Parity Generation

For a 4-bit dataword:

$$a_3 a_2 a_1 a_0$$

Parity bit is calculated as:

$$r_0 = a_3 \oplus a_2 \oplus a_1 \oplus a_0$$

where $\oplus$ denotes modulo-2 addition (XOR).

- Even number of 1s $\Rightarrow r_0 = 0$
- Odd number of 1s $\Rightarrow r_0 = 1$

Result

After adding r_0, the total number of 1s in the codeword becomes even.

Parity-Check Encoder and Decoder

Sender Side

$$r_0 = a_3 \oplus a_2 \oplus a_1 \oplus a_0$$

Codeword:

$$a_3 a_2 a_1 a_0 r_0$$

Receiver Side

The receiver computes the syndrome:

$$s_0 = b_3 \oplus b_2 \oplus b_1 \oplus b_0 \oplus q_0$$

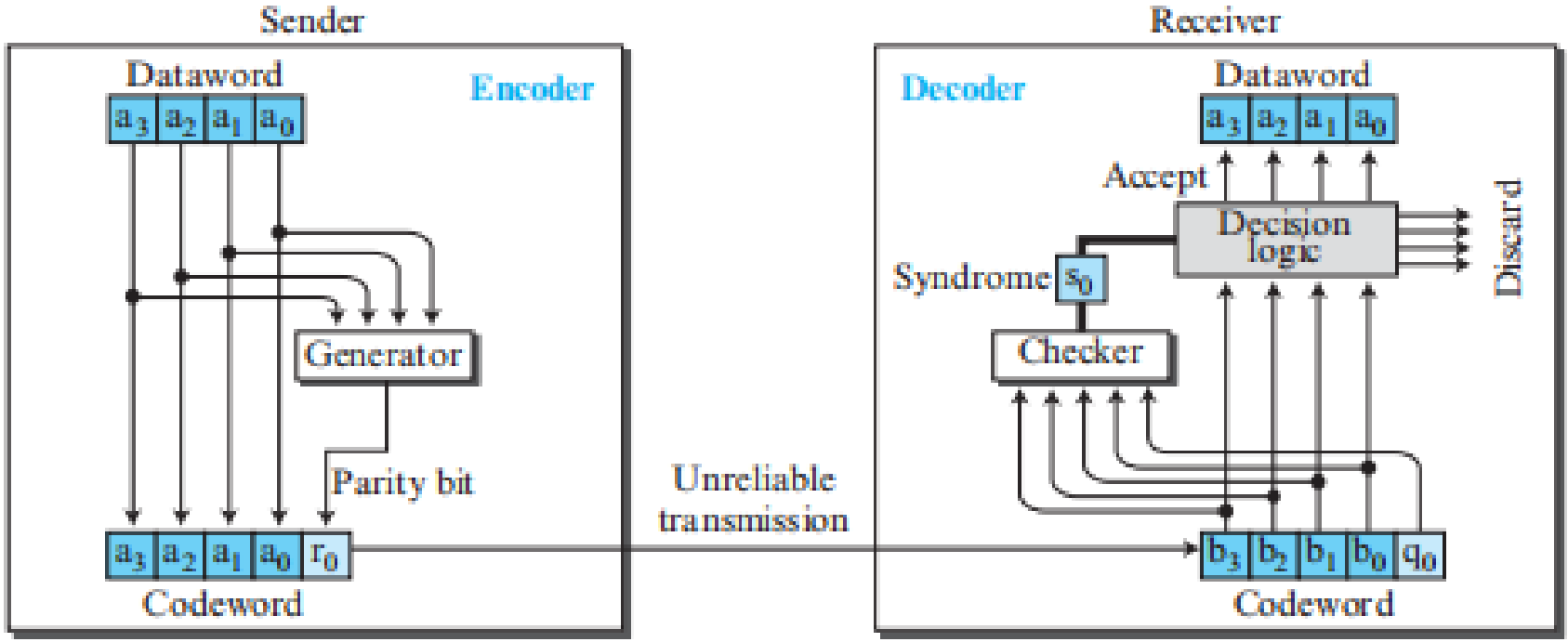
where received bits are

$$b_3, b_2, b_1, b_0, q_0$$

Decision

$$s_0 = 0 \Rightarrow \text{Accept Data}$$

Figure 5.11 *Encoder and decoder for simple parity-check code*



Parity-Check Encoder/Decoder Structure

- Encoder generates parity bit and appends it to data.
- Receiver recalculates parity using all received bits.
- Result is called the **syndrome**.
- Syndrome determines whether the received codeword is valid.

Example 5.7: Codeword Generation

Given dataword:

$$1011$$

Parity bit:

$$r_0 = 1 \oplus 0 \oplus 1 \oplus 1$$

$$r_0 = 1$$

Therefore the transmitted codeword is:

$$10111$$

Example 5.7: Case 1 (No Error)

Sent Codeword:

$$10111$$

Received Codeword:

$$10111$$

Syndrome:

$$s_0 = 0$$

Result

No error detected.
Dataword recovered:

$$1011$$

Example 5.7: Case 2 (Single-Bit Error)

Sent Codeword:

$$10111$$

Received Codeword:

$$10011$$

(one data bit corrupted)

Syndrome:

$$s_0 = 1$$

Result

Error detected.
Dataword is discarded.

Example 5.7: Case 3 (Parity Bit Error)

Sent Codeword:

$$10111$$

Received Codeword:

$$10110$$

(parity bit corrupted)

Syndrome:

$$s_0 = 1$$

Result

Error detected.
Even though data bits are correct, the receiver cannot determine which bit is wrong.

Example 5.7: Case 4 (Two-Bit Errors)

Received Codeword:

$$00110$$

Two bits are corrupted.

Syndrome:

$$s_0 = 0$$

Result

Receiver accepts the codeword.
Incorrect dataword:

$$0011$$

is generated.
Parity-check code cannot detect all even-numbered errors.

Example 5.7: Case 5 (Three-Bit Errors)

Received Codeword:

$$01011$$

Three bits are corrupted.

Syndrome:

$$s_0 = 1$$

Result

Error detected.
Dataword is discarded.

Capabilities of Parity-Check Code

Number of Errors	Detection
1	Detected
2	May Not Be Detected
3	Detected
5	Detected
Any Odd Number	Detected
Any Even Number	May Escape Detection

Important Result

A parity-check code can detect all odd-numbered errors but cannot reliably detect even-numbered errors.

Cyclic Codes

- Cyclic codes are a special class of **linear block codes**.
- They satisfy an additional property:

Cyclic Property

If a codeword is cyclically shifted (rotated), the resulting word is also a valid codeword.

- Example:

$$1011000 \rightarrow 0110001$$

- The last bit wraps around and becomes the first bit.
- Cyclic codes are widely used for error detection in communication networks.

Example of Cyclic Shift

Original Codeword:

$$1011000$$

After one cyclic left shift:

$$0110001$$

Bit relationship:

$$b_0 = a_6$$

$$b_1 = a_0,\ b_2 = a_1,\ b_3 = a_2$$

$$b_4 = a_3,\ b_5 = a_4,\ b_6 = a_5$$

Observation

The shifted word remains a valid codeword.

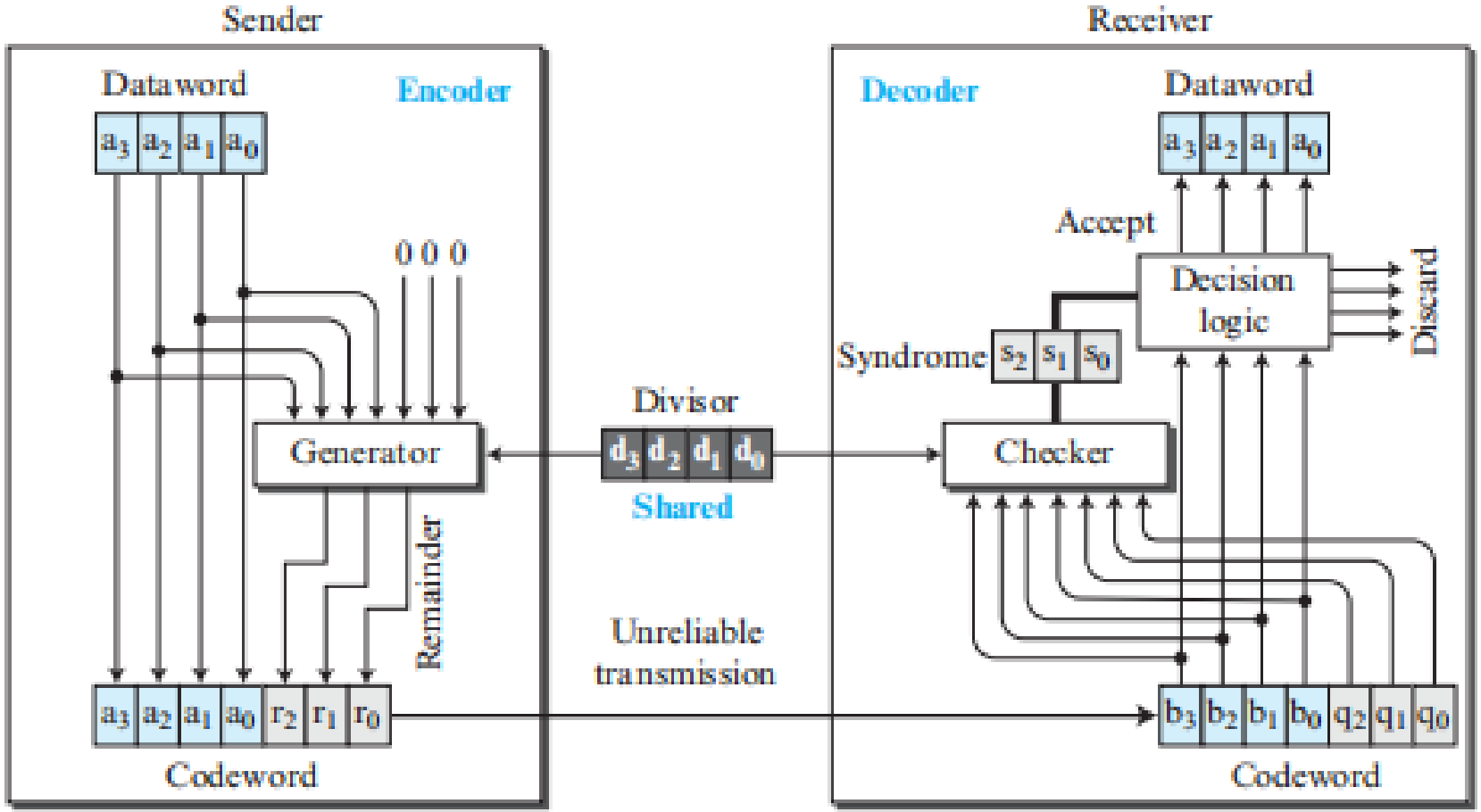
Cyclic Redundancy Check (CRC)

- CRC is the most widely used cyclic code in LANs and WANs.
- Used primarily for:
  - Error Detection
  - Data Integrity Verification
- CRC is based on:
  - Binary Polynomial Arithmetic
  - Modulo-2 Division
- Sender generates redundant check bits.
- Receiver recomputes and verifies them.

Key Idea

The receiver checks whether the received codeword is exactly divisible by the agreed generator.

Figure 5.12 *CRC encoder and decoder*



CRC Encoder and Decoder

- Encoder:
  - Adds redundant CRC bits.
  - Produces the codeword.
- Decoder:
  - Performs the same division.
  - Generates a syndrome.
  - Accepts or rejects the frame.

CRC Encoding Process

Given:

$$k = 4$$

$$n = 7$$

Number of CRC bits:

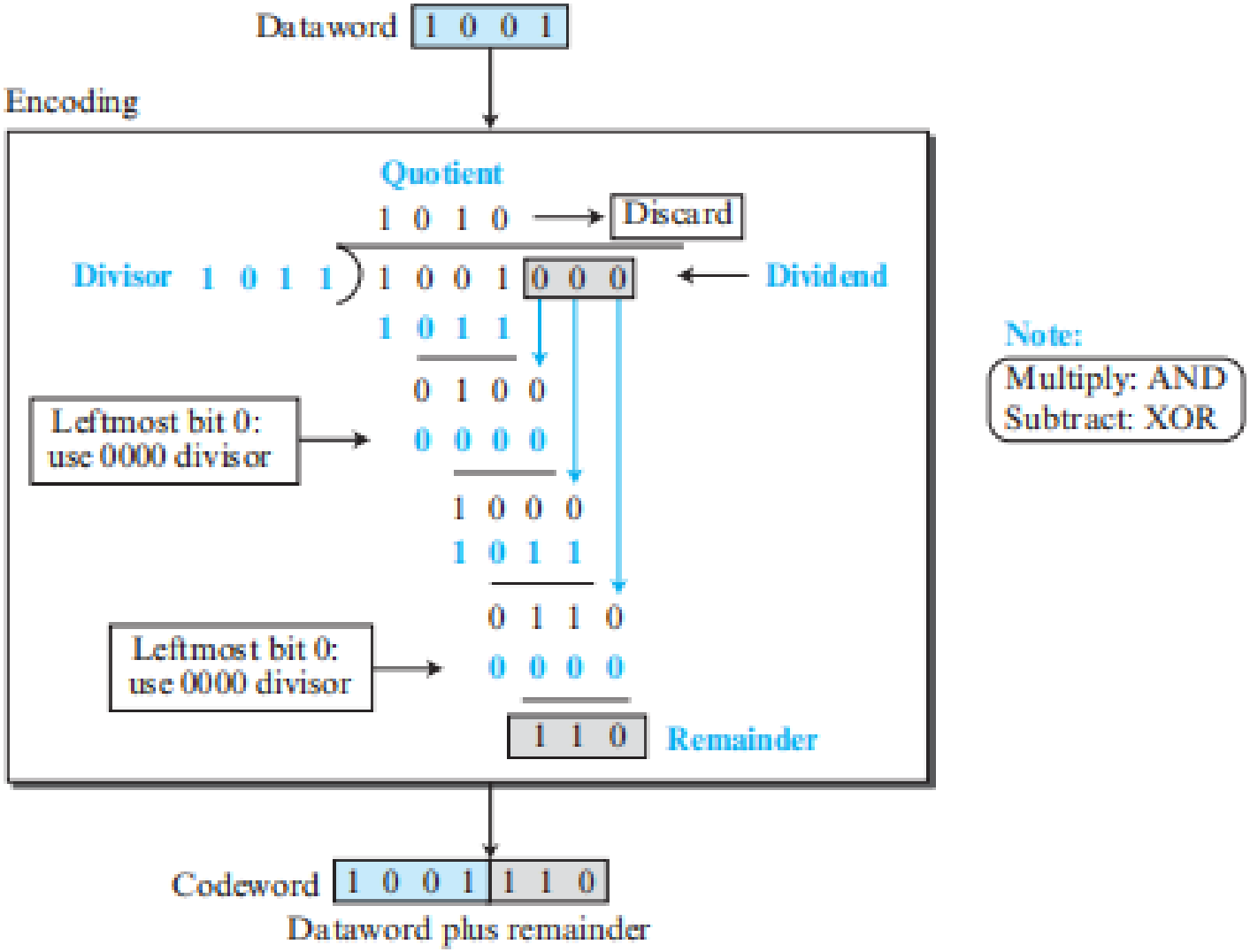
$$r = n - k = 3$$

Steps:

1. Take the dataword.
2. Append r zeros.
3. Divide using modulo-2 division.
4. Obtain remainder.
5. Append remainder to dataword.

$$\text{Codeword} = \text{Dataword} + \text{CRC Bits}$$

Figure 5.13 *Division in CRC encoder*



CRC Encoder Operation

- Dataword is augmented with zeros.
- Augmented word is divided by the generator.
- Quotient is discarded.
- Remainder becomes CRC check bits.
- Check bits are appended to form the codeword.

Modulo-2 Division

CRC uses binary division with XOR operations.

Rules:

$$0 \oplus 0 = 0$$

$$0 \oplus 1 = 1$$

$$1 \oplus 0 = 1$$

$$1 \oplus 1 = 0$$

Important

In modulo-2 arithmetic, addition and subtraction are identical and are performed using XOR.

CRC Decoding Process

At the receiver:

1. Receive the codeword.
2. Divide by the same generator.
3. Compute remainder (syndrome).
4. Analyze syndrome.

Decision:

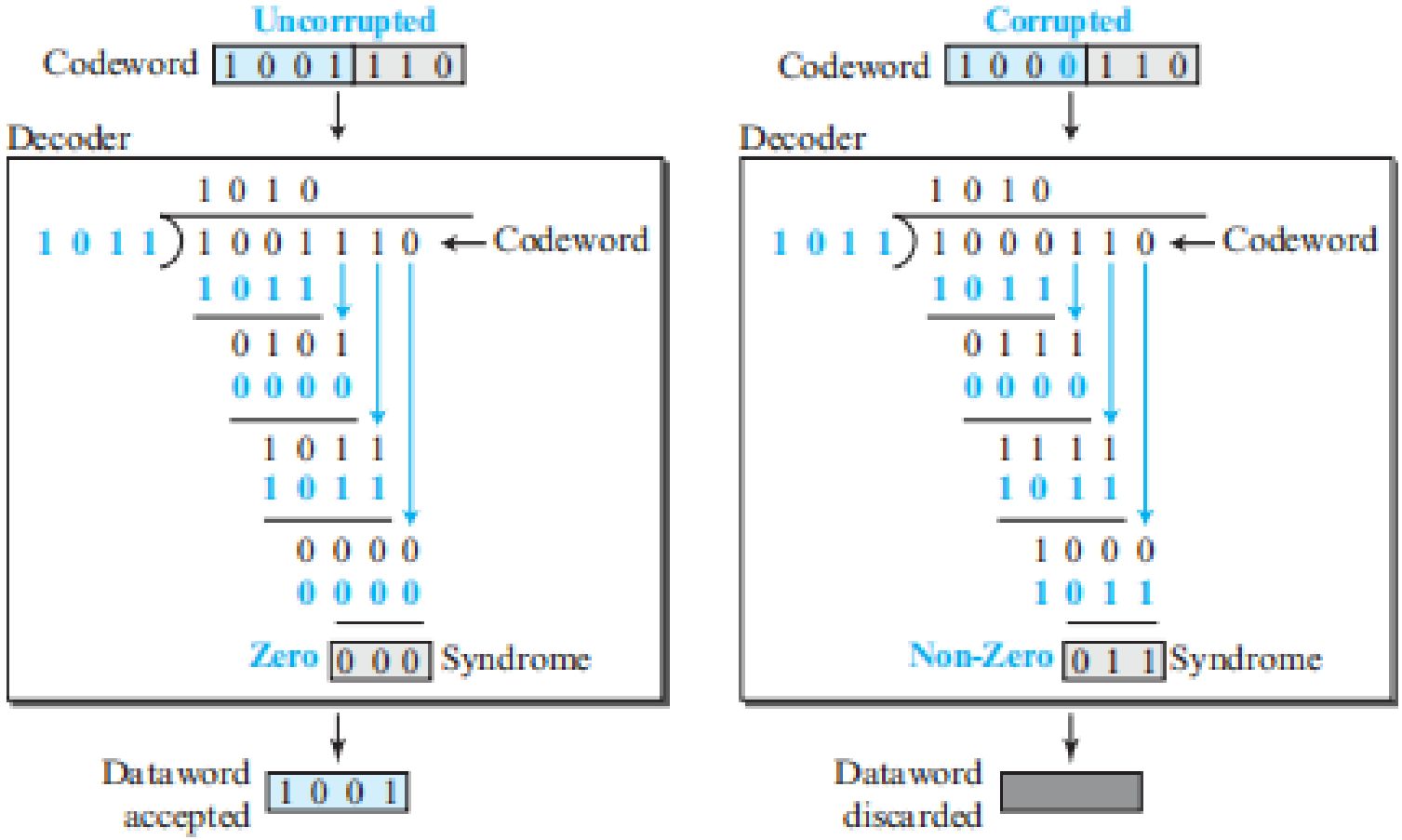
$$\text{Syndrome} = 000$$

$$\Rightarrow \text{Accept Frame}$$

$$\text{Syndrome} \neq 000$$

$$\Rightarrow \text{Error Detected}$$

Figure 5.14 *Division in the CRC decoder for two cases*



CRC Syndrome Checking

Two possible cases:

- No Error:

$$\text{Syndrome} = 000$$

- Error Present:

$$\text{Syndrome} \neq 000$$

Result

A nonzero syndrome indicates corruption during transmission.

CRC Generator (Divisor)

A CRC generator must satisfy:

1. Contain at least two bits.
2. First bit must be 1.
3. Last bit must be 1.

Example Generator:

$$1011$$

Why?

These properties ensure effective error-detection capability.

Table 5.4 *Standard polynomials*

Name	*Binary*	*Application*
CRC-8	100000111	ATM header
CRC-10	11000110101	ATM AAL
CRC-16	1000100000100001	HDLC
CRC-32	10000010011000001000111011011 0111	LANs

Standard CRC Generators

Common CRC Standards:

- CRC-8
- CRC-12
- CRC-16
- CRC-32

Note

The number indicates the polynomial degree.
CRC-32 uses a polynomial of degree 32 and a generator of 33 bits.

CRC Error Detection Performance

1. Detects all single-bit errors.
2. Detects most multiple-bit errors.
3. Detects burst errors with very high probability.

Single-Bit Errors

All valid CRC generators detect every single-bit error.

Detection of Odd Number of Errors

If the generator is divisible by:

$$(11)_2$$

then:

- All odd-numbered errors are detected.

Otherwise:

- Only some odd-numbered errors are detected.

Key Point

Many practical CRC generators are designed to detect all odd-numbered errors.

Burst Error Detection

Let:

$$L = \text{Burst Length}$$

$$r = \text{CRC Remainder Length}$$

CRC guarantees:

$$L \leq r$$

$$\Rightarrow 100\% \text{ Detection}$$

$$L = r + 1$$

$$\Rightarrow 1 - (0.5)^{r-1}$$

$$L > r + 1$$

Advantages of CRC

- Very strong error detection capability.
- Detects burst errors efficiently.
- Easy hardware implementation.
- Fast execution.
- Low overhead.
- Widely used in:
  - Ethernet
  - LANs
  - WANs
  - Storage Devices

Conclusion

CRC is the most popular error-detection technique used in modern communication networks.

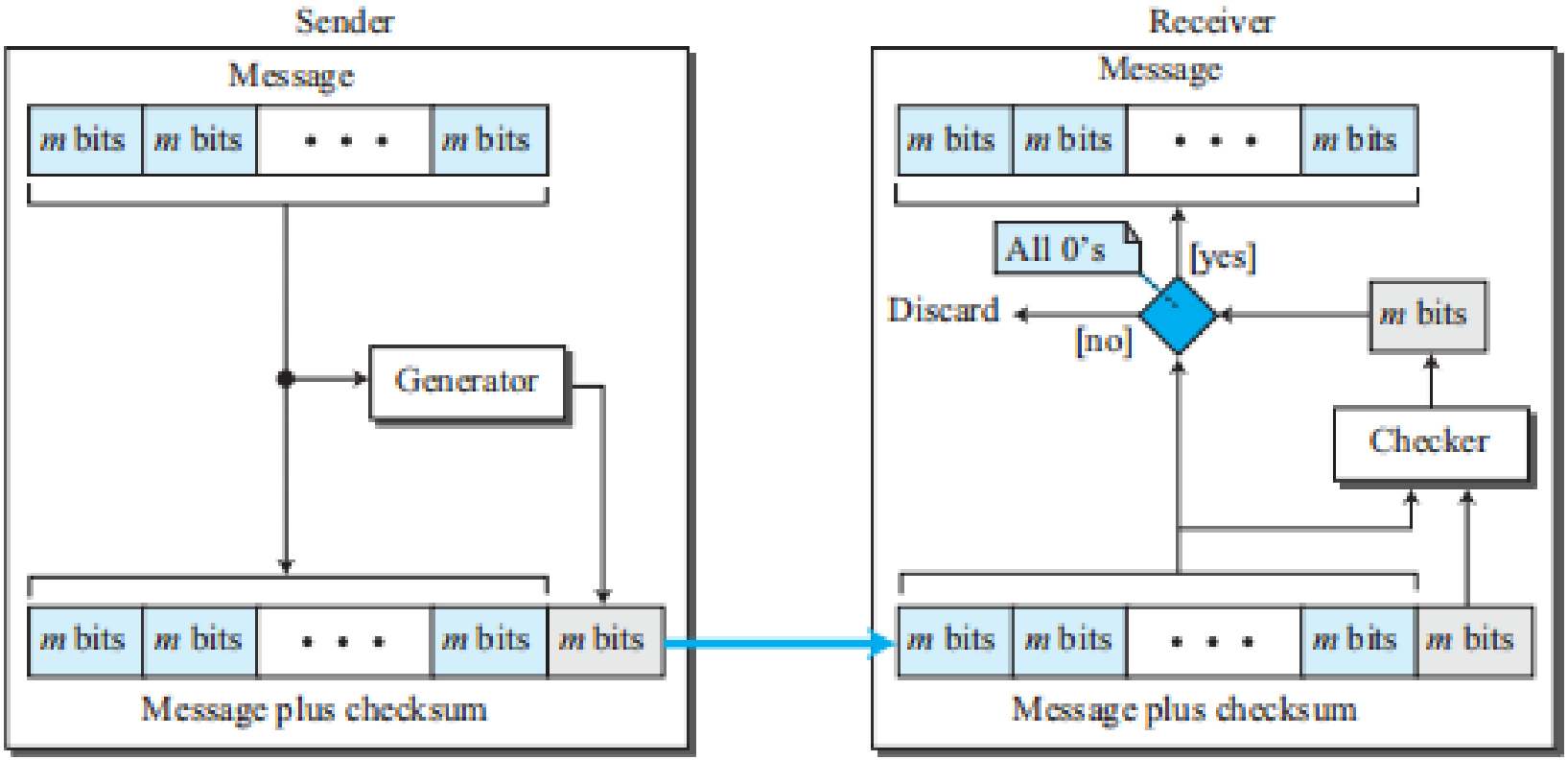
Checksum

- Checksum is an **error-detection technique** used for messages of any length.
- Commonly used in:
  - Network Layer (IP)
  - Transport Layer (TCP, UDP)
- Sender computes an additional value called the **checksum**.
- Receiver recalculates the checksum and verifies the message integrity.

Purpose

Detect errors that may occur during transmission.

Figure 5.15 *Checksum*



Checksum Process

1. Divide message into m-bit units.
2. Compute checksum from all units.
3. Send:

$$\text{Message} + \text{Checksum}$$

4. Receiver computes a new checksum.
5. If result is all zeros:

$$\text{Accept Message}$$

6. Otherwise:

$$\text{Discard Message}$$

Basic Idea Behind Checksum

- Sender calculates a value based on all message blocks.
- This value is transmitted along with the message.
- Receiver performs the same calculation.
- Matching results indicate no detectable error.

Key Idea

Checksum provides error detection using arithmetic operations on message blocks.

Example 5.8: Checksum Calculation

Suppose the message contains five 4-bit numbers:

$$(7,\ 11,\ 12,\ 0,\ 6)$$

Calculate the sum:

$$7 + 11 + 12 + 0 + 6 = 36$$

Sender transmits:

$$(7,\ 11,\ 12,\ 0,\ 6,\ 36)$$

where:

$$36$$

is the checksum value.

Example 5.8: Receiver Verification

Receiver receives:

$$(7,\ 11,\ 12,\ 0,\ 6,\ 36)$$

Receiver calculates:

$$7 + 11 + 12 + 0 + 6 = 36$$

Comparison:

$$36 = 36$$

Decision

Message is accepted because the computed sum matches the transmitted checksum.

Checksum Error Detection

If transmission errors occur:

$$(7,\ 11,\ 10,\ 0,\ 6,\ 36)$$

Receiver computes:

$$7 + 11 + 10 + 0 + 6 = 34$$

Comparison:

$$34 \neq 36$$

Result

Mismatch indicates an error in the message. The receiver discards the message.

Advantages and Limitations of Checksum

Advantages

- Simple implementation.
- Low computational cost.
- Suitable for large messages.

Limitations

- Detects many errors but not all.
- Less powerful than CRC.
- Cannot identify error location.
- Cannot correct errors.

Checksum vs CRC

Feature	**Checksum**	**CRC**
Method	Arithmetic Addition	Modulo-2 Division
Complexity	Simple	More Complex
Error Detection Strength	Moderate	Very High
Hardware Support	Limited	Excellent
Common Usage	IP, TCP, UDP	Ethernet, LANs, WANs

One's Complement Addition

- Simple summation may produce a result larger than the available bit size.
- One's complement arithmetic solves this problem by wrapping overflow bits.
- If the sum contains more than m bits:
  - Keep the rightmost m bits.
  - Add the extra leftmost bits back to the result.
- This process is called **end-around carry**.

Purpose

Represent numbers using only m bits while preserving arithmetic correctness.

Example 5.9: One's Complement Addition

From Example 5.8:

$$7 + 11 + 12 + 0 + 6 = 36$$

Binary representation:

$$36 = (100100)_2$$

Using 4-bit one's complement arithmetic:

$$100100$$

Split into:

$$10 + 0100$$

Add overflow bits:

$$0010 + 0100 = 0110$$

$$0110 = 6$$

Checksum Concept

- Sending the sum alone still requires comparison at the receiver.
- To simplify verification, the sender transmits the complement of the sum.
- This complement is called the **checksum**.

For an m-bit number:

$$\text{Checksum} = (2^m - 1) - \text{Sum}$$

Advantage

The receiver only checks whether the final result becomes zero.

One's Complement Representation

In one's complement arithmetic:

- Positive Zero:

$$0000$$

- Negative Zero:

$$1111$$

A number added to its complement produces:

$$1111$$

which represents negative zero.

Key Idea

If the final result is all 1s, the transmitted data is considered correct.

Example 5.10: Checksum Generation

From Example 5.9:

$$\text{Sum} = 6$$

Binary:

$$6 = (0110)_2$$

Complement:

$$1001$$

$$9 = (1001)_2$$

Sender transmits:

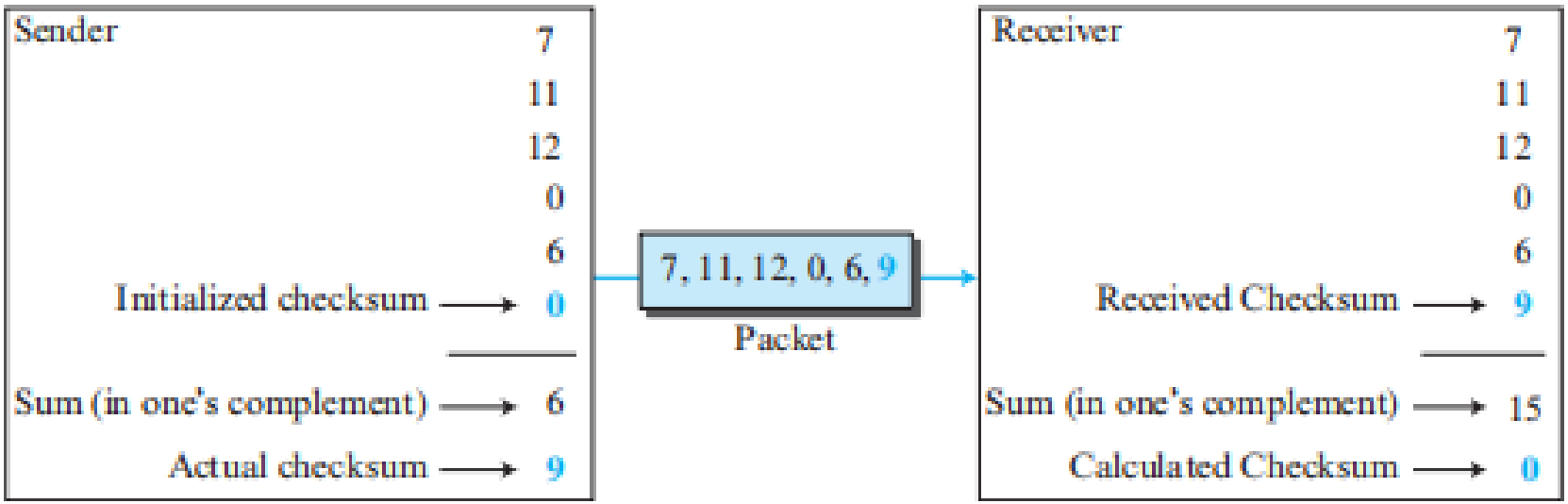
$$(7,\ 11,\ 12,\ 0,\ 6,\ 9)$$

where

$$9$$

is the checksum.

Figure 5.16 *Example 5.10*



Example 5.10: Receiver Verification

Receiver obtains:

$$(7,\ 11,\ 12,\ 0,\ 6,\ 9)$$

One's complement addition gives:

$$1111$$

which represents negative zero.
Complementing:

$$1111 \rightarrow 0000$$

Result

Checksum becomes zero.
No transmission error is detected.

Internet Checksum

Traditional Internet checksum uses:

$$16\text{-bit words}$$

Commonly used in:

- IP
- TCP
- UDP

Principle

All 16-bit words are added using one's complement arithmetic and the complement of the result becomes the checksum.

Table 5.5 *Procedure to calculate the traditional checksum*

Sender	*Receiver*
1. The message is divided into 16-bit words.	1. The message and the checksum is received.
2. The value of the checksum word is initially set to zero.	2. The message is divided into 16-bit words.
3. All words including the checksum are added using one's complement addition.	3. All words are added using one's complement addition.
4. The sum is complemented and becomes the checksum.	4. The sum is complemented and becomes the new checksum.
5. The checksum is sent with the data.	5. If the value of the checksum is 0, the message is accepted; otherwise, it is rejected.

Internet Checksum Procedure

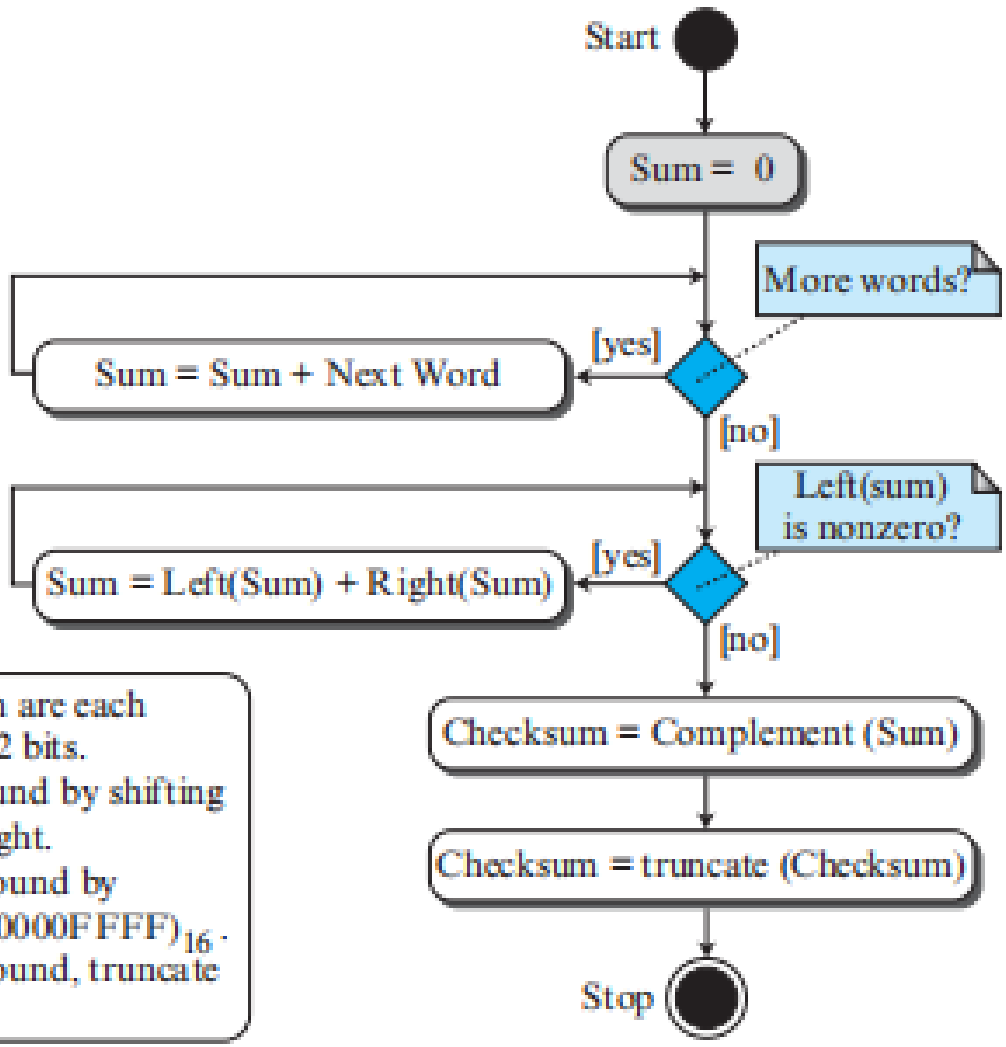
Sender:

1. Divide message into 16-bit words.
2. Add all words.
3. Compute complement.
4. Attach checksum.

Receiver:

1. Add all words including checksum.
2. Complement result.
3. Verify whether result is zero.

Figure 5.17 *Algorithm to calculate a traditional checksum*



Notes:

a. Word and Checksum are each 16 bits, but Sum is 32 bits.
b. Left(Sum) can be found by shifting Sum 16 bits to the right.
c. Right(Sum) can be found by ANDing Sum with $(0000FFFF)_{16}$.
d. After Checksum is found, truncate it to 16 bits.

Internet Checksum Algorithm

Algorithm Steps:

1. Add all data words.
2. Wrap overflow bits.
3. Compute one's complement.
4. Transmit checksum.

At the receiver:

1. Repeat the same process.
2. Verify checksum value.

Performance of Traditional Checksum

Advantages:

- Simple implementation.
- Low computational overhead.
- Suitable for large messages.

Limitations:

- Weaker than CRC.
- Cannot detect some compensating errors.
- Two modified words may leave checksum unchanged.

Observation

Checksum provides moderate error detection but is less reliable than CRC.

Problem with Traditional Checksum

Traditional checksum is:

- Not weighted.
- Independent of word position.

Example:
If two 16-bit words swap positions:

$$A, B \rightarrow B, A$$

the checksum remains unchanged.

Limitation

Data transposition errors may not be detected.

Improved Checksum Techniques

To overcome weaknesses of traditional checksum:

1. Fletcher Checksum
2. Adler Checksum

These techniques:

- Apply weighting to data.
- Improve error-detection capability.
- Detect many transposition errors.

Fletcher Checksum

Fletcher checksum assigns weights based on position.
Two versions:

- 8-bit Fletcher
- 16-bit Fletcher

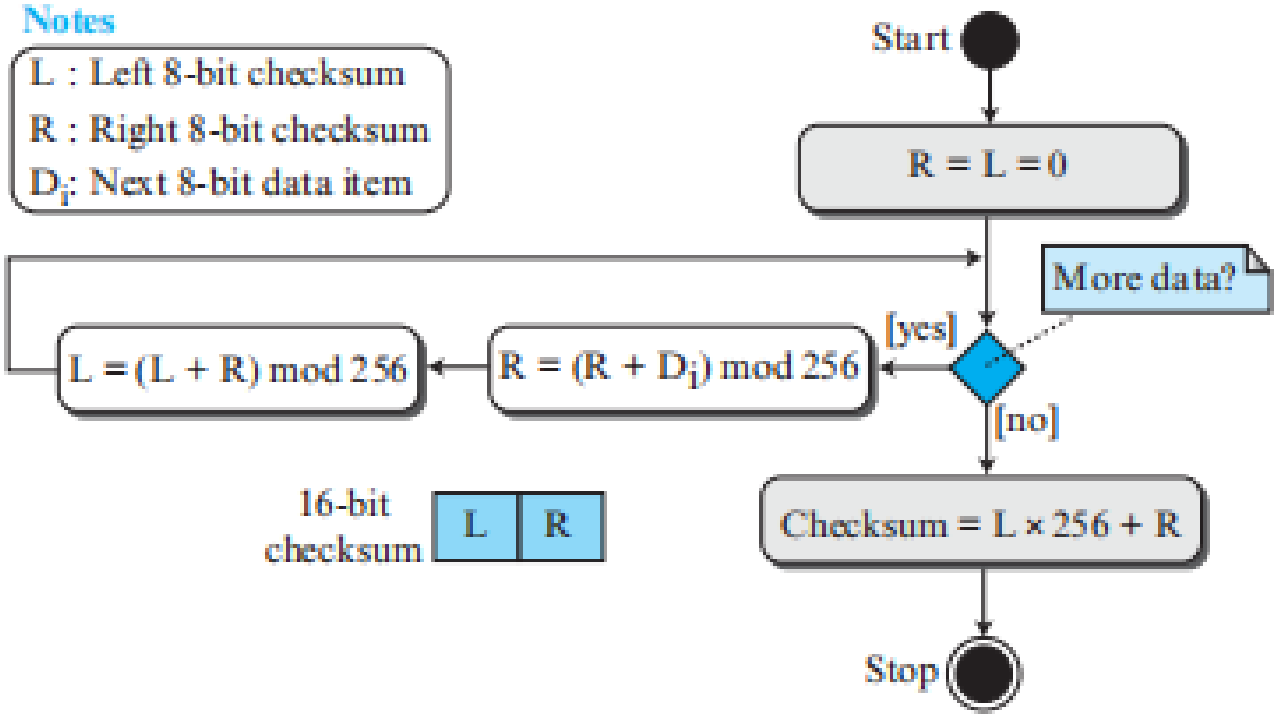
Characteristics:

- Uses two accumulators:

$$L,\ R$$

- Computation performed modulo arithmetic.
- Better detection than traditional checksum.

Figure 5.18 *Algorithm to calculate an 8-bit Fletcher checksum*



8-bit Fletcher Checksum

Features:

- Operates on 8-bit data items.
- Produces:

$$16\text{-bit checksum}$$

- Calculations performed modulo:

$$256$$

- Two accumulators:

$$L = \text{Running Sum}$$

$$R = \text{Weighted Sum}$$

16-bit Fletcher Checksum

Features:

- Operates on 16-bit data items.
- Produces:

$$32\text{-bit checksum}$$

- Calculations performed modulo:

$$65536$$

- Higher error-detection capability than 8-bit Fletcher.

Adler Checksum

Adler checksum is another weighted checksum technique.
Characteristics:

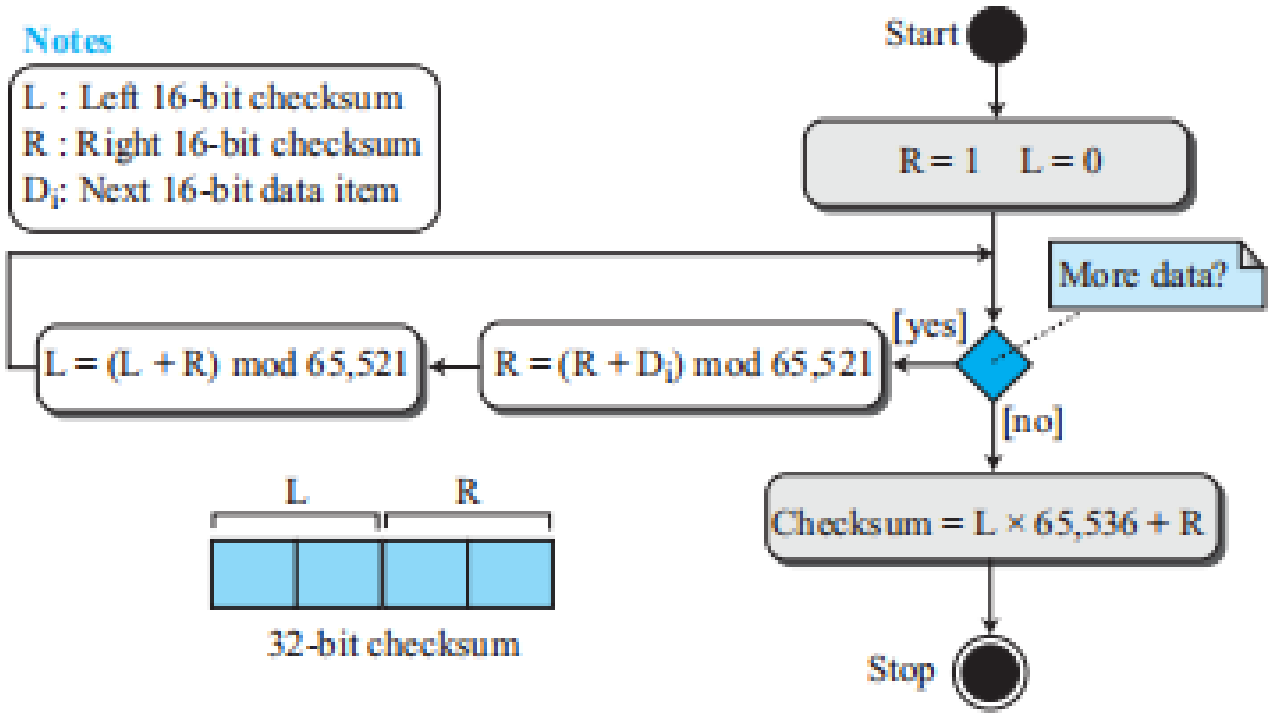
- Produces:

$$32\text{-bit checksum}$$

- Operates on bytes.
- Similar to Fletcher checksum.
- Uses a prime modulus:

$$65521$$

Figure 5.19 *Algorithm to calculate an Adler checksum*



Adler vs Fletcher Checksum

Feature	**Fletcher**	**Adler**
Data Unit	Byte / Word	Byte
Checksum Size	16 or 32 bits	32 bits
Modulus	256 / 65536	65521 (Prime)
Weighted Calculation	Yes	Yes
Error Detection	Better than Traditional	Better than Traditional

Summary of Checksum Techniques

- Traditional Checksum
  - Simple
  - Fast
  - Limited detection capability
- Fletcher Checksum
  - Weighted checksum
  - Better detection
- Adler Checksum
  - Uses prime modulus
  - Improved reliability

Conclusion

Modern protocols increasingly prefer CRC because it offers stronger error detection than checksum techniques.